

LLMsVerifier

LLMsVerifier

検証。監視。最適化。

概要

LLMsVerifierは、複数のプロバイダーにまたがる大規模言語モデル（LLM）を検証・監視・最適化するエンタープライズグレードのプラットフォームです。「私のコードが見えますか？」という必須の検証テストに基づいて構築されており、実際に機能することが証明されたモデルのみが「使用可能」とマークされ、エクスポートされます。

短い説明

複数のプロバイダーにまたがるLLMを検証・ベンチマーク・監視・最適化するGoプラットフォーム。すべてのモデルは使用前に必須のコード可視性テストに合格する必要がある、その後レイテンシー、ストリーミング、関数呼び出し、ビジョン、埋め込みのチェックを実施し、検証済みの設定のみをAIやCLIツールにエクスポートします。

長い説明

LLMsVerifierは、複数のプロバイダーにまたがるLLMのパフォーマンスを検証・監視・最適化する包括的なプラットフォームです。その核となる原則は必須検証であり、これに関しては一切妥協しません。どのモデルも「使用可能」とマークされるか、エクスポートされた設定に組み込まれる前に、「私のコードが見えますか？」というテストに合格する必要があります。このテストでは、プロバイダーに対して実際のHTTPリクエストを送信し、その応答を分析して、単なるもっともらしいエコーではなく、真の理解が示されているかを確認します。入力内容を実際に認識し理解できないモデルは、「使用可能」フラグを獲得することはありません。この関門をクリアした後、Verifier Engineは存在確認、応答性、レイテンシー、ストリーミング、関数呼び出し、ビジョン、embeddingsなど、包括的な能力テストを実行し、Reporter Engineがその結果をマークダウンやJSONレポートに変換して、実行可能な形で提供します。

システムはモジュール式かつイベント駆動型で、Verifier Engine、Reporter Engine、Configuration Managerをコアとし、CLI、TUI、Web、REST、APIといったインターフェースを提供します。検証にとどまらず、高度なレイヤーではLLMを活用したタスク分解のためのSupervisor/Workerパターン、非常に長いセッションでも崩壊しないようスライディングウィンドウとLLMによる要約を用いたコンテキスト管理、クラウドベースのチェックポイント、サーキットブレーカーとレイテンシーベースのルーティングを備えたフェイルオーバーシステムを実装しています。周辺インフラは本番環境に対応した構成となっており、pub/subイベントバス、cronスケジューリング、価格・制限の検出、RAG用のvectorデータベース、エクスポートシステムを備えています。また、生成されたプロバイダーやモデルにはすべて「(llmsvd)」という接尾辞が付加されるため、検証済みの出力は一目で識別可能で、未検証のものと混同されることはありません。さらに、検証済みのモデルのみが、OpenCode、Crush、Claude CodeなどのAIやCLIツール向けにエクスポートされた設定に書き込まれます。本番環境で実際に必要とされる運用ツールも標準装備されており、Docker/Kubernetes/Helmによるデプロイメント、Prometheus/Grafanaによる監視、LDAP/SSO、SQLCipher暗号化ストレージなどが含まれます。

開発の背景

設定ファイルのチェックだけでは信頼性に欠けるからです。APIのキーが期限切れになったり、モデルが非推奨になったりすることがありますが、設定ファイルからは実際のレイテンシーやエラー、モデルが本当に入力内容を認識し理解しているかどうかは分かりません。LLMsVerifierは、「設定ファイルに記載されているから動くはず」という考え方を排し、実際に正しく応答することが証明されたモデルのみを「使用可能」とマークし、エクスポートするという証拠主義を採用しています。

コンテンツ

なぜこれはゲームチェンジャーなのか

LLMのフリートに「信頼できる」という、設定が省略によって嘘をつくことの多いこの領域ではめったに与えられない評価をもたらします。設定したモデルが機能することを願うのではなく、実際の検証を通過したモデルだけが稼働し、監視、フェイルオーバー、検証済みのみのエクスポートによって、証明から本番環境までのループが閉じられます。Helixエコシステム内では、LLMのモデル、プロバイダー、検証メタデータに対する唯一の信頼できる情報源となり、他のサービス（HelixTranslateなど）はこれに基づいてルーティングされるため、プラットフォーム全体が「今どのモデルが実際に機能しているのか」という一つの正直な答えを継承します。各チームがそれぞれの希望的観測を維持する必要はありません。

革新的なポイント

- 「私のコードが見えますか？」の必須検証 – モデルが使用可能になる前に通過しなければならない、実際のHTTPベースの理解度チェック。製品の最大の特徴であり、未検証のものが紛れ込む余地を与えない理由です。
- 検証済みのみの設定エクスポート – AIやCLIツール向けに生成される設定には、検証を通過したモデルのみが含まれるため、壊れたモデルがこっそりと再導入されることはありません。
- (llmsvd) ブランディングサフィックスシステム – 生成されたプロバイダーやモデルにはすべて追跡可能なサフィックスが付与され、出力がどこに伝播しても検証済みの来歴が可視化されます。
- 多数のCLIエージェント・プロバイダーにわたる機能検出 – ストリーミング形式 (SSE、WebSocket、JSONL、EventStream)、圧縮、キャッシュ動作を仮定せずに指紋認証します。
- レジリエントなフェイルオーバー – サーキットブレーカー、レイテンシーベースのルーティング (初回トークンまでの時間が閾値を超えると再ルーティング)、ヘルスプローブ、重み付きトラフィック分割により、個々のプロバイダーが不安定になってもフリートは応答性を維持します。
- 長時間の自律稼働 – スーパーバイザー／ワーカー分割パターンに加え、チェックポイントとメモリ統合により、コンテキストが枯渇することなく長時間のセッションを維持します。
- **RAG／vector-DBとの連携**によるグラウンデッドコンテキストの強化。

最大の技術的課題とその解決策

- モデルが「設定されている」だけでなく「実際に機能する」ことを証明すること。これが最大の目的であり、最も難しい部分でした。実際のAPI呼び出しを行い、肯定的な理解が得られるかどうかを分析する必須のコード可視性テストと、幅広い機能テストによって解決。さらに、検証を通過しなかったものは一切エクスポートしないことで、設定ではなく証明が本番環境へのゲートとなります。
- 多数の不安定なサードパーティプロバイダーにわたる信頼性の確保。プロバイダーの不安定性を通常の状態として扱うフェイルオーバーオーケストレーターによって解決。N回の失敗がM秒以内に発生したらプロバイダーを劣化とマークするサーキットブレーカー、遅いエンドポイントを避けるレイテンシーベースのルーティング、回復を確認する定期的なヘルスチェック、コスト効率と高品質モデルのバランスを取る重み付きルーティングを採用。
- 非常に長時間の自律セッションの維持。大きな作業を扱いやすい単位に分割するスーパーバイザー／ワーカー分割パターン、中断しても進捗を維持するクラウドストレージへの定期的なチェックポイント、階層化されたコンテキスト管理 (スライディングウィンドウ+LLM要約+RAG) により、モデルがトークンに埋もれることなくスレッドを維持します。
- プロバイダーの乱立。多数のプロバイダー固有のGoアダプターを一つの共通インターフェースの背後に隠蔽し、実際のエンドポイントを中央

で管理することで解決。プロバイダーの追加は局所的な変更にとどまり、コードベース全体に波及しません。

コンテンツ

テックスタック

- **Go** — 並行処理に優れているため、コアプラットフォーム言語として採用。マルチスレッドのVerifier Engineを駆動し、複数のモデルを並列に検証可能。周辺サービスも統括する。
- **Gin** — REST APIサーバーとして採用。JWT認証、レートリミット、WebSocket/SSEエンドポイントを担当。
- **SQLite + SQLCipher** — データベースレベルの暗号化を備えた組み込みストレージとして採用。検証データ（キー、結果）は機密性が高く、デフォルトで保存時暗号化が必要なため。
- **Redis** — キャッシュ層として採用。頻繁にアクセスされる検証やメタデータのルックアップを高速化。
- **RabbitMQ + Kafka** — イベント駆動型アーキテクチャを支えるために採用。メッセージングとストリーミングにより、プラットフォーム全体でプロデューサーとコンシューマーを疎結合化。
- **gRPC + Protocol Buffers** — 型安全なサービス間通信とコンポーネント間のイベント転送を実現するために採用。
- **QUIC / HTTP-3 (quic-go)** — 最新のトランスポートプロトコルをサポートするために採用（リポジトリのドキュメントではHTTP/3プロバイダーの利用可能性が限定的と明記されており、これは「提供機能」であって「普遍的な主張」ではない）。
- **JWT + LDAP/NTLM** — エンタープライズ認証として採用。既存の企業ID（SSO/SAML/OIDC）にシームレスに統合可能（ドキュメント上の記載）。
- **Viper（設定）、Logrus（ロギング）、Brotli/compress（圧縮）** — 運用基盤：柔軟な設定管理、構造化ログ、ペイロード圧縮。
- **Angular** — Webシングルページアプリケーションとして採用。検証とモニタリングの視覚的なフロントドア。
- **Python + JavaScript SDK** — クライアントチームに第一級のアクセス手段を提供。OpenAPI/Swaggerによるドキュメント整備。
- **Docker、Kubernetes、Helm** — 本番環境へのデプロイメントを支えるために採用。ヘルスモニタリングとオートスケーリングにより、検証フリートは現代的なサービスと同様にスケール可能。
- **Prometheus + Grafana** — メトリクスとダッシュボードを提供。プラットフォーム自体の健全性を、監視対象のモデルと同じように可視化。
- **Testify (Go) + node -test/jsdom (Web)** — GoコアとWebフロントエンドにわたる多層的なテストを実現。

ステータスと率直な注意点

- ステータス：ベータ版。Goのソースコードは実際のHTTP検証を実装済み（検証を設定のみで行うとする古いドキュメントは理想的な記述であり、現状とは異なる。コードが正確な情報源）。
- ライセンス：未定。READMEにはMITと記載されているが、DockerfileのラベルにはApache-2.0と記載。公開前に解決が必要。
- プロバイダー数：READMEには「12のアダプター」と記載されているが、プロバイダーディレクトリには約26のエントリーが存在。「12以上 / 開発中のものも含む」と捉えること。多数の「FINAL/COMPLETE」ステータスファイルが存在するが、コード、ドキュメント、go.modが信頼できる情報源。
- リポジトリはvasic-digital組織に属しているが、機能的にはHelix LLMインフラストラクチャクラスターの信頼レイヤーとして機能。

優先度：Helixプライマリ（LLMインフラストラクチャクラスター。LLM/プロバイダー/検証メタデータの唯一の情報源）。HelixTrackに次ぐ優先度。